Digital System Design
Term Project Report

Team Number: 6	
Student Number	Student Name
21101263	김범준
22101261	도현서
21101339	이우림
22101495	정현우

Submission Date : 2026. 06. 10.

Contents

1. Abstract

This project implements and evaluates three FPGA SRCNN accelerators: recursive SRCNN_42, recursive SRCNN_88, and streamline SRCNN_84. The design focus was latency reduction under FPGA resource limits. We first used channel-wise parallel computation, observed its resource cost through SRCNN_88, applied 2-pixel horizontal execution to SRCNN_42, and finally designed SRCNN_84 for streamline layer overlap.

All RTL simulations matched the fixed-point reference software with zero mismatches for three 150x150 images. At 100 MHz, total latency from i_start to the final o_done was 275,459 cycles for SRCNN_42, 413,183 cycles for SRCNN_88, and 70,297 cycles for SRCNN_84. Demo outputs were also evaluated with PSNR against the ground-truth images.

2. Objective

The objective is to compare FPGA SRCNN accelerators by channel parallelism, memory mapping, PE scheduling, dataflow, latency, resources, timing, correctness, and PSNR.

TABLE 1 **Design Configurations**

DESIGN	CHANNEL CONFIGURATION	ARCHITECTURE TYPE	MAIN DESIGN PURPOSE
SRCNN_88	8 / 8	Recursive	Channel-wise parallel baseline and resource-cost reference
SRCNN_42	4 / 2	Recursive	2-pixel horizontal latency reduction using available resource margin
SRCNN_84	8 / 4	Streamline	Layer-overlapped processing for lower total latency

그림 1 Design Configurations

The three designs compare wider channel-wise PE parallelism, horizontal pixel parallelism, and streamline overlap.

3. Pre-survey

Before RTL design, we checked three project-specific points. First, 01_Reference_SW was used to understand Q8.8 arithmetic, bias/scale handling, saturation, clipping, and reference output order. The RTL target was bit-level agreement, not only visual similarity.

Second, the provided input/reference/demo files defined the verification path. The testbenches compare final outputs, write counts, write addresses, and selected

intermediate layer outputs. The demo environment reconstructs images and evaluates PSNR.

Third, we surveyed hardware mapping choices for SRCNN-like convolution: channel-wise PE parallelism, BRAM width/depth packing, line-buffer reuse for 3x3 windows, fixed-point inference, and CNN data reuse [1]-[5]. These shaped the packed activation buffers, scan-window line buffers, PE array sizing, and streamline padding modules.

4. Design process

4.1 Overall Interface and Verification Target

All designs follow the skeleton interface. Top modules are srcnn42_top, srcnn88_top, and srcnn84_top; the wrapper packs four 16-bit output pixels into one 64-bit demo word. o_done is asserted only after the final image is completed.

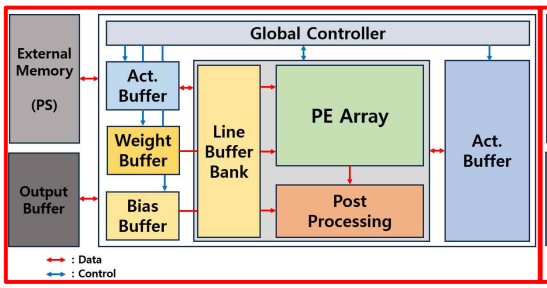

그림 2 srcnn42 & srcnn88 Diagram

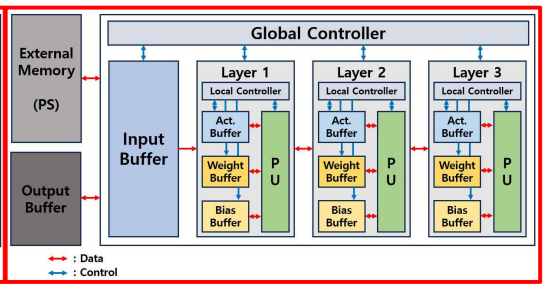

그림 3 srcnn84 Diagram

4.2 Recursive Controller and FSM

The recursive designs execute layers sequentially while reusing a shared convolution datapath. The controller selects image index, layer, input source, activation memory path, weight/bias set, write address, and output path.

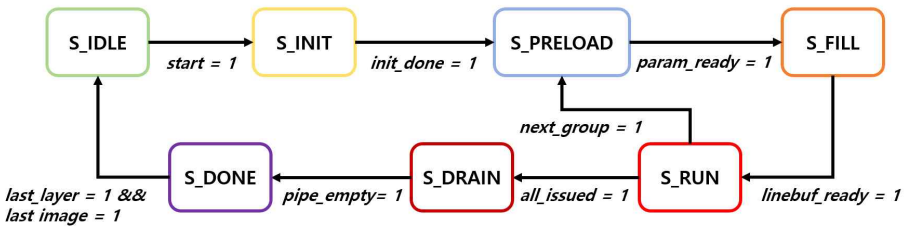

그림 4 Recursive Architecture State Diagram (FSM)

INIT resets counters. PRELOAD prepares row/window state. FILL supplies padded pixels or packed activations until the 3x3 window is ready. RUN issues MAC operations, DRAIN waits for PE/post-processing latency, and DONE generates a one-cycle completion pulse.

4.3 Recursive Improvements: SRCNN_88 and SRCNN_42

SRCNN_88 represents the channel-wise parallel recursive direction. Layer 2 assigns one PE slot to each output-channel/input-channel pair. This removes channel folding in the main computation, but also explains why SRCNN_88 has the highest LUT, BRAM, and DSP usage.

SRCNN_42 uses the same recursive control style but exploits resource margin differently. Instead of widening channels, it duplicates the base PE group horizontally and processes adjacent columns for activation-producing layers. The line buffer exposes adjacent 3x3 windows, and activation memory stores packed horizontal pixel pairs.

4.4 Streamline Improvement: SRCNN_84

SRCNN_84 improves latency through layer overlap rather than recursive replay. Layer 1, layer 2, and layer 3 have local line buffers and PE arrays. Between layers, srcnn84_pad_insert regenerates the padded stream so the next layer can consume valid outputs as a continuous feature stream.

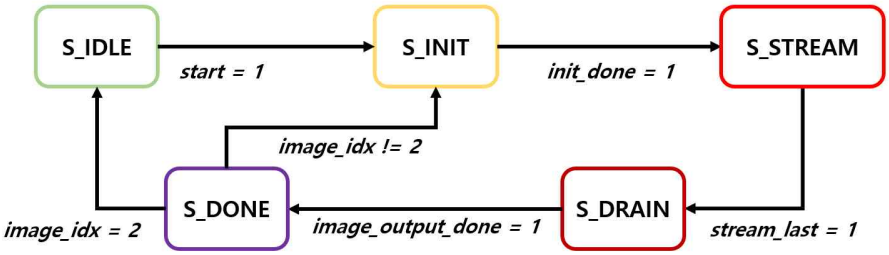

그림 5 Streamline Architecture State Diagram (FSM)

It uses more layer-local datapath hardware, but avoids full layer-by-layer activation replay. As a result, SRCNN_84 has the lowest measured total latency.

4.5 Memory, Line Buffer, and PE Design

Recursive activations are stored in packed BRAM words to reduce width/depth fragmentation. SRCNN_88 packs channel values, while SRCNN_42 also packs horizontal pixel pairs.

The recursive line buffer shifts a scan window horizontally during RUN, avoiding a large dynamic column-selection mux. SRCNN_42 adds adjacent-column taps for 2-pixel operation. The streamline line buffer stores previous padded rows and horizontal delay registers.

Each PE performs one signed 16-bit Q8.8 3x3 MAC. The final PE registers nine multiplication results and accumulates them next cycle, reducing unnecessary pipeline

registers while meeting 100 MHz timing. Post-processing applies bias, scaling, saturation, and clipping.

4.6 Verification Flow

RTL outputs are compared with fixed-point reference outputs. The testbench checks final values, write addresses, write counts, and selected internal layer outputs.

TABLE 2 Verification Datasets

DATASET	INPUT FILE	REFERENCE OUTPUT
Basic set	input_text1_2_3.txt	expected_output_1_2_3.txt
Additional set	input_text51_89_64.txt	expected_output_51_89_64.txt

그림 6 Verification Datasets

5. Analysis and Evaluation

5.1 Experimental Conditions

TABLE 3 Experimental Conditions

ITEM	CONDITION
Image size	150 x 150
Number of images	3
Data format	16-bit Q8.8 fixed point
Kernel size	3 x 3
Clock	10 ns / 100 MHz
Functional reference	01_Reference_SW fixed-point output
Image-quality metric	Y-channel PSNR

그림 7 Experimental Conditions

5.2 Functional Verification Results

TABLE 4 Functional Verification

DESIGN	ARCHITECTURE	TOTAL LATENCY (CYCLES)	TOTAL LATENCY (US)	WRITES	MISMATCHES
SRCNN_42	Recursive, 2-pixel, channel-wise	275,459	2754.59	67,500	0
SRCNN_88	Recursive, channel-wise	413,183	4131.83	67,500	0
SRCNN_84	Streamline	70,297	702.97	67,500	0

Latency is measured from the rising i_start request to the final o_done after all three 150 x 150 images.

그림 8 Functional Verification

Total latency is measured from the rising i_start request to the final o_done after all three images. For recursive designs, this total latency is the meaningful metric; steady-state throughput is not emphasized because the architecture processes layer by layer.

5.3 Hardware Utilization and Timing

TABLE 5 Hardware Utilization and Timing

DESIGN	LUT	LUT %	FF	FF %	BRAM	BRAM %	DSP	DSP %	WNS	TIMING
SRCNN_42	57,820	49.37	69,404	29.63	73	50.69	145	11.62	0.589	Met
SRCNN_88	104,330	89.08	129,691	55.37	112	77.78	577	46.23	1.106	Met
SRCNN_84	40,980	34.99	79,463	33.92	33	22.92	397	31.81	2.550	Met

URAM usage is 5 blocks (7.81%) for all three designs and is omitted here to keep the table compact for report capture.

그림 9 Hardware Utilization

SRCNN_88 uses the most resources because layer 2 has the largest channel-wise PE array. SRCNN_42 reduces recursive latency through horizontal pixel parallelism. SRCNN_84 achieves the lowest latency and BRAM use through streamline overlap and local buffering.

5.4 PSNR Evaluation

The actual demo result for image set 1/2/3 was:

TABLE 6 Demo PSNR Results

DESIGN	TEST IMAGE	BICUBIC PSNR (DB)	SRCNN PSNR (DB)	IMPROVEMENT (DB)
SRCNN_42	test_1	26.69	31.15	+4.46
SRCNN_88	test_1	26.69	31.68	+4.99
SRCNN_84	test_1	26.69	31.82	+5.13

그림 10 PSNR Results

The hardware SRCNN output improves PSNR for all three test images, and RTL simulation confirms that output values match the reference software.

6. Differentiation strategies (Your Own Contribution)

Our contributions beyond a direct skeleton implementation are:

1. We implemented and compared three SRCNN hardware architectures with different optimization goals: channel-wise recursive, 2-pixel recursive, and streamline.
2. We used packed activation memory to improve BRAM utilization in recursive designs.
3. We designed scan-window line buffers that reduce large dynamic column-selection logic.
4. We applied horizontal 2-pixel execution in SRCNN_42 after confirming resource margin.
5. We built SRCNN_84 with layer-local line buffers and padding insertion for streamline overlap.
6. We tuned PE pipeline depth based on timing margin and verified all designs with both basic and additional image sets.

※ References

[1] C. Dong, C. C. Loy, K. He, and X. Tang, "Image Super-Resolution Using Deep Convolutional Networks," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 38, no. 2, pp. 295-307, 2016, doi: 10.1109/TPAMI.2015.2439281.

[2] C. Dong, C. C. Loy, and X. Tang, "Accelerating the Super-Resolution Convolutional Neural Network," European Conference on Computer Vision (ECCV), pp. 391-407, 2016, doi: 10.1007/978-3-319-46475-6_25.

[3] J.-W. Chang, K.-W. Kang, and S.-J. Kang, "An Energy-Efficient FPGA-Based Deconvolutional Neural Networks Accelerator for Single Image Super-Resolution," IEEE Transactions on Circuits and Systems for Video Technology, vol. 30, no. 1, pp. 281-295, 2020, doi: 10.1109/TCSVT.2018.2888898.

[4] D. D. Lin, S. S. Talathi, and V. S. Annapureddy, "Fixed Point Quantization of Deep Convolutional Networks," Proceedings of the 33rd International Conference on Machine Learning (ICML), pp. 2849-2858, 2016.

[5] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," IEEE Journal of Solid-State Circuits, vol. 52, no. 1, pp. 127-138, 2017, doi: 10.1109/JSSC.2016.2616357.